Penetration Test

Luke

Report of Findings

HTB Certified Penetration Testing Specialist (CPTS) Exam Report

Candidate Name: Lakshya Rastogi

Luke Ltd.

Version: 1.0

CONFIDENTIAL

Table of Contents

1 Statement of Confidentiality 4

2 Engagement Contacts 5

3 Executive Summary 6

3.1 Approach 6

3.2 Scope 6

3.3 Assessment Overview and Recommendations 6

4 Network Penetration Test Assessment Summary 8

4.1 Summary of Findings 8

5 Internal Network Compromise Walkthrough 10

5.1 Detailed Walkthrough 10

6 Remediation Summary 12

6.1 Short Term 12

6.2 Medium Term 12

6.3 Long Term 12

7 Technical Findings Details 14

Cleartext Storage of Administrative Credentials Leading to Full System Compromise 14

Hardcoded Credentials in Web Application Source Code 18

Broken Access Control and Sensitive Data Exposure via API 20

Anonymous FTP Access and Information Disclosure 22

A Appendix 24

A.1 Finding Severities 24

A.2 Host & Service Discovery 25

A.3 Subdomain Discovery 26

A.4 Exploited Hosts 27

A.5 Compromised Users 28
A.6 Changes/Host Cleanup 29
A.7 Flags Discovered 30

1 Statement of Confidentiality

The contents of this document have been developed by Hack The Box. Hack The Box considers the contents of this document to be proprietary and business confidential information. This information is to be used only in the performance of its intended use. This document may not be released to another vendor, business partner or contractor without prior written consent from Hack The Box. Additionally, no portion of this document may be communicated, reproduced, copied or distributed without the prior consent of Hack The Box.

The contents of this document do not constitute legal advice. Hack The Box's offer of services that relate to compliance, litigation or other legal interests are not intended as legal counsel and should not be taken as such. The assessment detailed herein is against a fictional company for training and examination purposes, and the vulnerabilities in no way affect Hack The Box external or internal infrastructure.

2 Engagement Contacts

Luke Contacts		
Contact	**Title**	**Contact Email**

Assessor Contact		
Assessor Name	**Title**	**Assessor Contact Email**
Lakshya Rastogi	Security Consultant	Lakshyarastogi483@gmail.com

3 Executive Summary

Luke Ltd. ("Luke" herein) contracted Lakshya Rastogi to perform a Network Penetration Test of Luke's externally facing network to identify security weaknesses, determine the impact to Luke, document all findings in a clear and repeatable manner, and provide remediation recommendations.

3.1 Approach

Lakshya Rastogi performed testing under a "Black Box" approach from , to without credentials or any advance knowledge of Luke's externally facing environment with the goal of identifying unknown weaknesses. Testing was performed from a non-evasive standpoint with the goal of uncovering as many misconfigurations and vulnerabilities as possible. Testing was performed remotely from Lakshya Rastogi's assessment labs. Each weakness identified was documented and manually investigated to determine exploitation possibilities and escalation potential. Lakshya Rastogi sought to demonstrate the full impact of every vulnerability, up to and including internal domain compromise. If Lakshya Rastogi were able to gain a foothold in the internal network, Luke as a result of external network testing, Luke allowed for further testing including lateral movement and horizontal/vertical privilege escalation to demonstrate the impact of an internal network compromise.

3.2 Scope

The scope of this assessment was one external IP address, two internal network ranges, the Luke Active Directory domain, and any other Active Directory domains owned by Luke discovered if internal network access were achieved.

In Scope Assets

Host/URL/IP Address	Description
10.10.X.X	Luke

3.3 Assessment Overview and Recommendations

During the penetration test against Luke, Lakshya Rastogi identified 4 findings that threaten the confidentiality, integrity, and availability of Luke's information systems. The findings were categorized by severity level, with 0 of the findings being assigned a critical-risk rating, 2 high-risk, 2 medium-risk, and 0 low risk. There were also 0 informational finding related to enhancing security monitoring capabilities within the internal network.

1. Engagement Overview

During this engagement, a comprehensive vulnerability assessment and penetration test was conducted against the target server, "Luke." The primary objective was to identify security weaknesses that could be exploited by a malicious actor, evaluate the effectiveness of existing security controls, and determine the potential business impact of a successful breach.

2. Overall Risk Posture: CRITICAL

The assessment revealed severe, chainable vulnerabilities that allowed an unauthenticated, remote attacker to achieve total administrative compromise (root access) of the target server. Due to the complete loss of system confidentiality, integrity, and availability, the overall risk rating for this environment is designated as **CRITICAL**. Immediate remediation efforts are required.

3. Key Findings & Business Impact

The attack path highlighted significant systemic flaws in access control, credential management, and API security:

- **Insecure Credential Storage:** Hardcoded database credentials were discovered in publicly accessible web source code. This fundamental flaw provided the initial foothold into the application and bypasses standard authentication mechanisms.
- **Broken API Access Control:** A backend application API failed to adequately restrict user access. By leveraging the previously discovered credentials, it was possible to query the API and extract the plaintext passwords of all registered users on the system. In a production environment, this represents a catastrophic data breach and severe compliance violation.
- **Exposed Administrative Interfaces:** Internal management directories were left exposed without proper authorization checks. This led to the discovery of a plaintext configuration file containing the master administrative credentials for the server's control panel (Ajenti).
- **Full System Takeover:** Utilizing the exposed master credentials, full root-level execution was achieved. This grants an attacker absolute control over the underlying operating system, allowing for data exfiltration, service disruption, or the establishment of persistent backdoors for further network exploitation.

4. Strategic Recommendations

To secure the environment and mitigate future risk, the organization must immediately rotate all compromised credentials and restrict access to exposed management interfaces. Moving forward, it is imperative to eliminate the practice of storing plaintext credentials in source code and configuration files by implementing a centralized secrets management solution. Furthermore, all internal APIs must be refactored to enforce strict Role-Based Access Control (RBAC) and data minimization, ensuring users can only access information strictly necessary for their authorized roles.

Luke should create a remediation plan based on the Remediation Summary section of this report, addressing all high findings as soon as possible according to the needs of the business. Luke should also consider performing periodic vulnerability assessments if they are not already being performed. Once the issues identified in this report have been addressed, a more collaborative, in-depth Active Directory security assessment may help identify additional opportunities to harden the Active Directory environment, making it more difficult for attackers to move around the network and increasing the likelihood that Luke will be able to detect and respond to suspicious activity.

4 Network Penetration Test Assessment Summary

Lakshya Rastogi began all testing activities from the perspective of an unauthenticated user on the internet. Luke provided the tester with network ranges but did not provide additional information such as operating system or configuration information.

4.1 Summary of Findings

During the course of testing, Lakshya Rastogi uncovered a total of 4 findings that pose a material risk to Luke's information systems. Lakshya Rastogi also identified 0 informational finding that, if addressed, could further strengthen Luke's overall security posture. Informational findings are observations for areas of improvement by the organization and do not represent security vulnerabilities on their own. The below chart provides a summary of the findings by severity level.

In the course of this penetration test **2 High** and **2 Medium** vulnerabilities were identified:

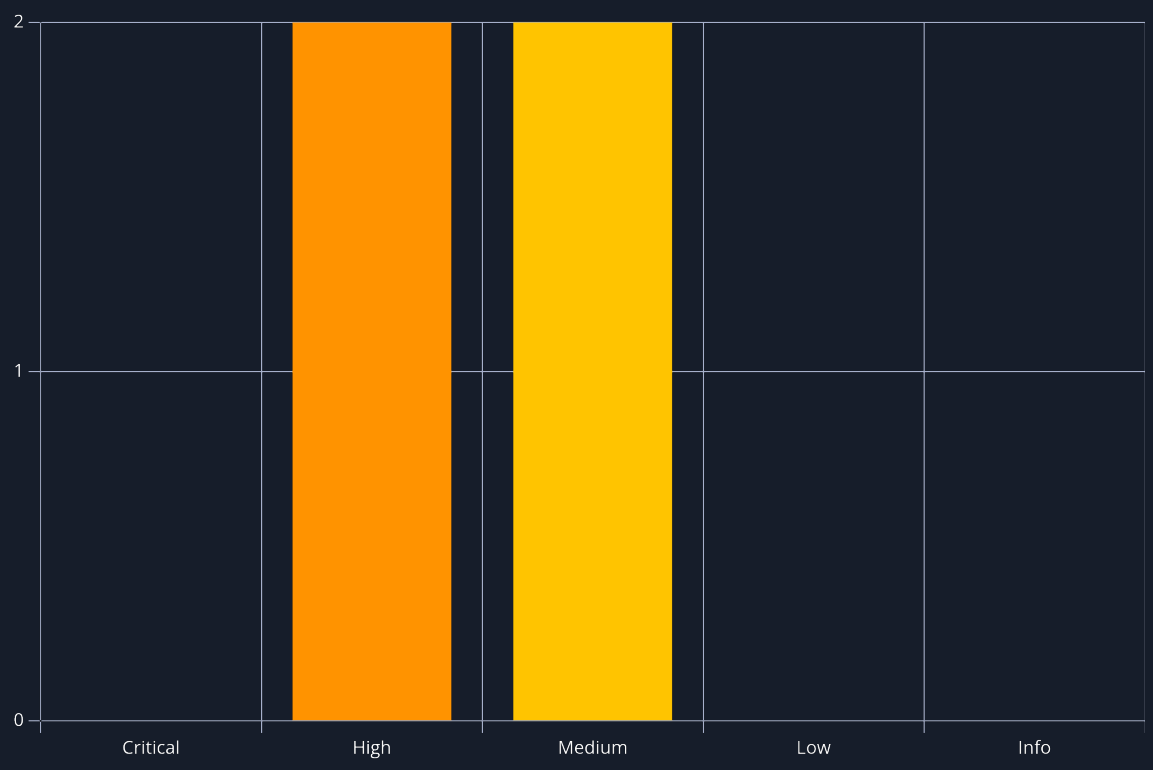


Figure 1 - Distribution of identified vulnerabilities

Below is a high-level overview of each finding identified during testing. These findings are covered in depth in the Technical Findings Details section of this report.

#	Severity Level	Finding Name	Page
1	8.8 (High)	Cleartext Storage of Administrative Credentials Leading to Full System Compromise	14

#	Severity Level	Finding Name	Page
2	7.5 (High)	Hardcoded Credentials in Web Application Source Code	18
3	6.5 (Medium)	Broken Access Control and Sensitive Data Exposure via API	20
4	5.3 (Medium)	Anonymous FTP Access and Information Disclosure	22

5 Internal Network Compromise Walkthrough

During the course of the assessment Lakshya Rastogi was able gain a foothold via the external network, move laterally, and compromise the internal network, leading to full administrative control over the Luke Active Directory domain. The steps below demonstrate the steps taken from initial access to compromise and does not include all vulnerabilities and misconfigurations discovered during the course of testing. Any issues not used as part of the path to compromise are listed as separate, standalone issues in the Technical Findings Details section, ranked by severity level. The intent of this attack chain is to demonstrate to Luke the impact of each vulnerability shown in this report and how they fit together to demonstrate the overall risk to the client environment and help to prioritize remediation efforts (i.e., patching two flaws quickly could break up the attack chain while the company works to remediate all issues reported). While other findings shown in this report could be leveraged to gain a similar level of access, this attack chain shows the initial path of least resistance taken by the tester to achieve domain compromise.

5.1 Detailed Walkthrough

Lakshya Rastogiperformed the following to fully compromise the Luke domain.

Phase 1: Initial Reconnaissance & Enumeration

- **Port Scanning:** Initial network scanning identified four critical open ports: 21 (FTP), 80 (HTTP Web Server), 3000 (HTTP API), and 8000 (HTTP Ajenti Management Panel).
- **Service Enumeration (FTP):** Investigation of the FTP service on port 21 revealed it was misconfigured to allow anonymous logins.
- **Information Disclosure:** Authenticating as `anonymous` allowed the extraction of an internal file (`for_chihiro.txt`), confirming the presence of active users and providing initial environmental context.

Phase 2: Web Enumeration & Credential Discovery

- **Directory Fuzzing:** Moving to the web server on port 80, aggressive directory and file fuzzing (using `ffuf`) was performed to map the attack surface.
- **Source Code Exposure:** The fuzzing revealed a publicly accessible `config.php` file. Because the server failed to interpret the PHP code, rendering the raw file via a web browser (or `curl`) exposed hardcoded plaintext database credentials (`root` / `Zk6heYCyv6ZE9Xcg`).

Phase 3: API Exploitation & Credential Harvesting

- **API Mapping:** Concurrent enumeration on port 3000 identified a custom web API with `/login` and `/users` endpoints.
- **Credential Reuse & Authentication Bypass:** Operating on the assumption of password reuse, the password discovered in `config.php` was tested against the `/login` endpoint using the username `admin`. This successfully authenticated and returned a valid JSON Web Token (JWT).
- **Broken Object Level Authorization (BOLA):** The obtained JWT was injected into the authorization header of a `GET` request to the `/users` endpoint. The API failed to restrict data access, returning a complete JSON array of all registered system users (Admin, Derry, Yuri, Dory) alongside their plaintext passwords.

Phase 4: Lateral Movement

- **Targeted Authentication:** With a newly harvested list of valid credentials, access was systematically tested across the remaining discovered services.
- **Accessing Internal Portals:** The credentials for the user "Derry" (`Derry` / `rZ86wwLvx7jUxtch`) successfully authenticated to a restricted management directory located at `http://[Target_IP]/management` on port 80.

Phase 5: Privilege Escalation & System Compromise

- **Directory Listing Vulnerability:** The `/management` endpoint suffered from improper access controls, presenting an open directory index.
- **Extraction of Administrative Secrets:** Navigating this directory revealed a `config.json` file. Analyzing this file uncovered the plaintext administrative credentials (`root` / `KpMaswqhS5ETTy9Z`) for the Ajenti server control panel.
- **Full System Compromise:** These credentials were used to authenticate to the Ajenti panel on port 8000. As Ajenti operates with elevated privileges, this provided a root-level web terminal and file manager, resulting in total administrative control over the target system.

6 Remediation Summary

As a result of this assessment there are several opportunities for Luke to strengthen its internal network security. Remediation efforts are prioritized below starting with those that will likely take the least amount of time and effort to complete. Luke should ensure that all remediation steps and mitigating controls are carefully planned and tested to prevent any service disruptions or loss of data.

6.1 Short Term

Short-Term Objectives (Immediate Action Required)

These are high-priority "quick wins" that rely on configuration changes to immediately disrupt the attack chains discovered during the assessment.

- **Rotate Compromised Credentials:** Immediately change all compromised passwords, prioritizing the Ajenti `root` password, database credentials, and all user accounts (Admin, Derry, Yuri, Dory). Ensure the new passwords are long, complex, and unique.
- **Disable Anonymous FTP:** Reconfigure the FTP service on port 21 to explicitly deny `anonymous` logins.
- **Restrict Directory Access & Listing:** Disable directory listing (Index of /) across the web server. Immediately restrict access to the `/management` directory using web server access controls (e.g., `.htaccess`, IP allowlisting, or requiring strong authentication) to protect exposed configuration files like `config.json`.
- **Fix Web Server Parsing:** Correct the web server configuration on port 80 to ensure `.php` files are executed by the PHP engine rather than being served as raw, downloadable text.

6.2 Medium Term

Medium-Term Objectives (Within 30–90 Days)

These items require development effort to fix underlying application logic and improve the security posture of the custom services.

- **Remediate API Data Exposure:** Refactor the `/users` endpoint on the port 3000 API to enforce data minimization. Under no circumstances should the API return password hashes or plaintext passwords in its JSON responses.
- **Implement Object-Level Authorization:** Fix the Broken Object Level Authorization (BOLA) vulnerability on the API. Ensure that authenticated users can only query information that their specific role explicitly grants them access to.
- **Remove Hardcoded Secrets:** Refactor the web application source code to completely remove hardcoded credentials (like those found in `config.php`). Transition to using environment variables or a local encrypted keystore.

6.3 Long Term

Long-Term Objectives (Strategic & Architectural)

These recommendations focus on programmatic changes to organizational processes to prevent these types of vulnerabilities from being introduced in the future.

- **Implement a Secrets Management Solution:** Deploy a centralized, secure secrets vault (such as HashiCorp Vault or a cloud-native equivalent) to dynamically inject database and administrative credentials into applications at runtime, eliminating the need for `config.json` or `config.php` credential storage entirely.
- **Secure Code Review Integration:** Integrate automated static analysis security testing (SAST) and manual secure code reviews into the development pipeline to catch hardcoded secrets and API authorization flaws before they reach production.
- **Principle of Least Privilege:** Conduct a comprehensive audit of all services (such as the Ajenti control panel and web server processes) to ensure they are running with the absolute minimum operating system privileges necessary, rather than executing as `root`.

7 Technical Findings Details

1. Cleartext Storage of Administrative Credentials Leading to Full System Compromise - High

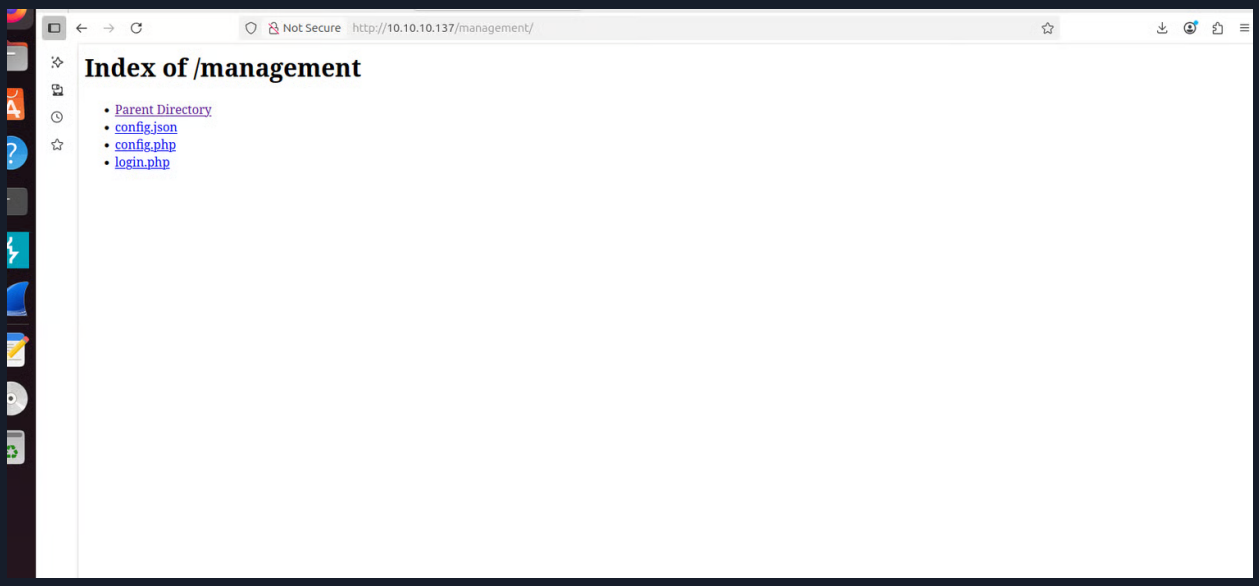
CWE	CWE-312 - Cleartext Storage of Sensitive Information
CVSS 3.1	8.8 / CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H
Root Cause	**Vulnerability Description** The web directory located at `/management` fails to adequately protect sensitive configuration files from authenticated users. After logging into the management portal using credentials obtained from the vulnerable API (Finding 3), it was discovered that directory listing is enabled and a `config.json` file is directly accessible. This file contains the complete, plaintext configuration for the Ajenti server administration panel running on port 8000, including the master `root` username and password.
Impact	**Impact** The exposure of the `config.json` file completely compromises the confidentiality and integrity of the underlying server. By extracting the plaintext credentials, an attacker can authenticate to the Ajenti management panel as the `root` user. Since Ajenti is a server administration tool designed to manage the operating system, this grants the attacker total control over the server, allowing for arbitrary command execution, file manipulation, and further network pivoting.
Remediation	**Remediation** • **Secure File Storage:** Never store sensitive configuration files containing plaintext passwords within the web root directory. Move these files to a secure location on the filesystem that cannot be served by the web application. • **Hash Sensitive Secrets:** Administrative passwords must be securely hashed using strong cryptographic algorithms (such as Argon2 or bcrypt) rather than being stored in cleartext. • **Disable Directory Listing:** Ensure that directory listing (Index of /) is disabled on the web server to prevent attackers from discovering hidden or unlinked files. • **Enforce Principle of Least Privilege:** Ensure that users accessing the `/management` directory do not have permissions to read files that are strictly intended for the system administration service.
References	-

Finding Evidence

Technical Details

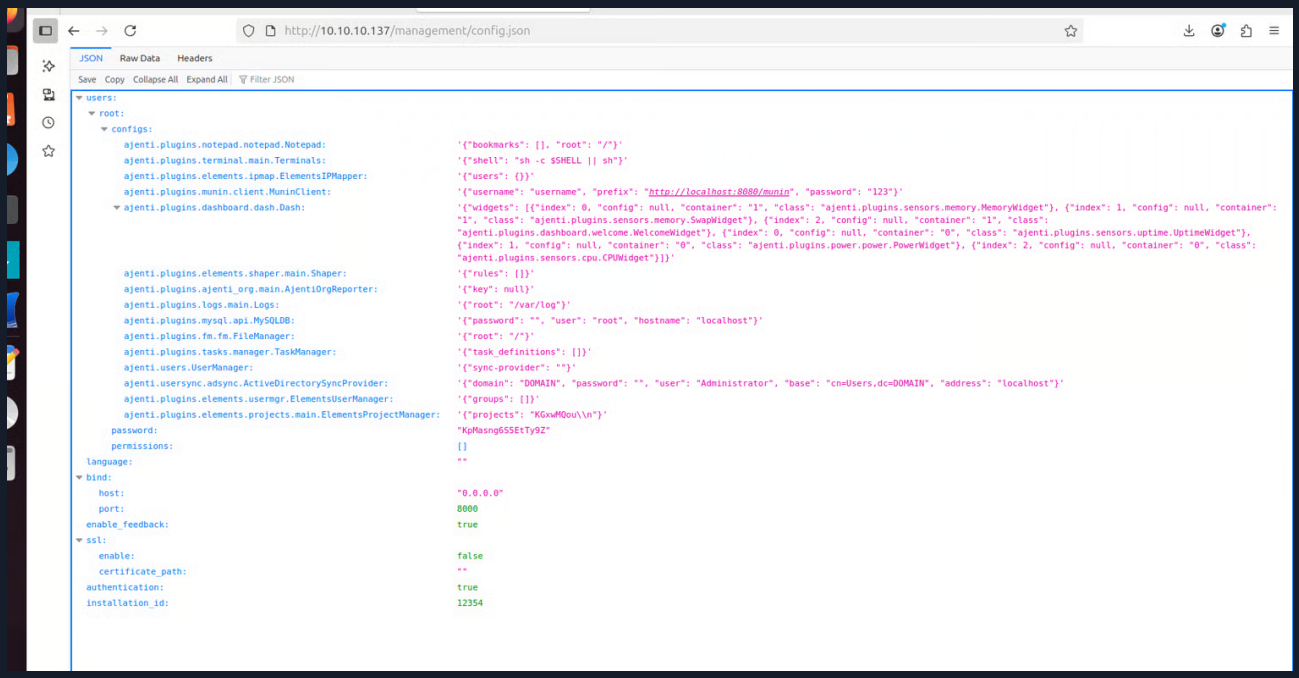
An attacker can leverage compromised lower-level credentials to access the management portal and extract the Ajenti root credentials.

1. Navigate to the `/management` directory on port 80 and log in using the credentials for the user `Derry` (`Derry` / `rZ86wwLvx7jUxtch`) discovered in the previous API enumeration step.
2. Upon successful authentication, the server presents a directory listing containing `config.json`, `config.php`, and `login.php`.

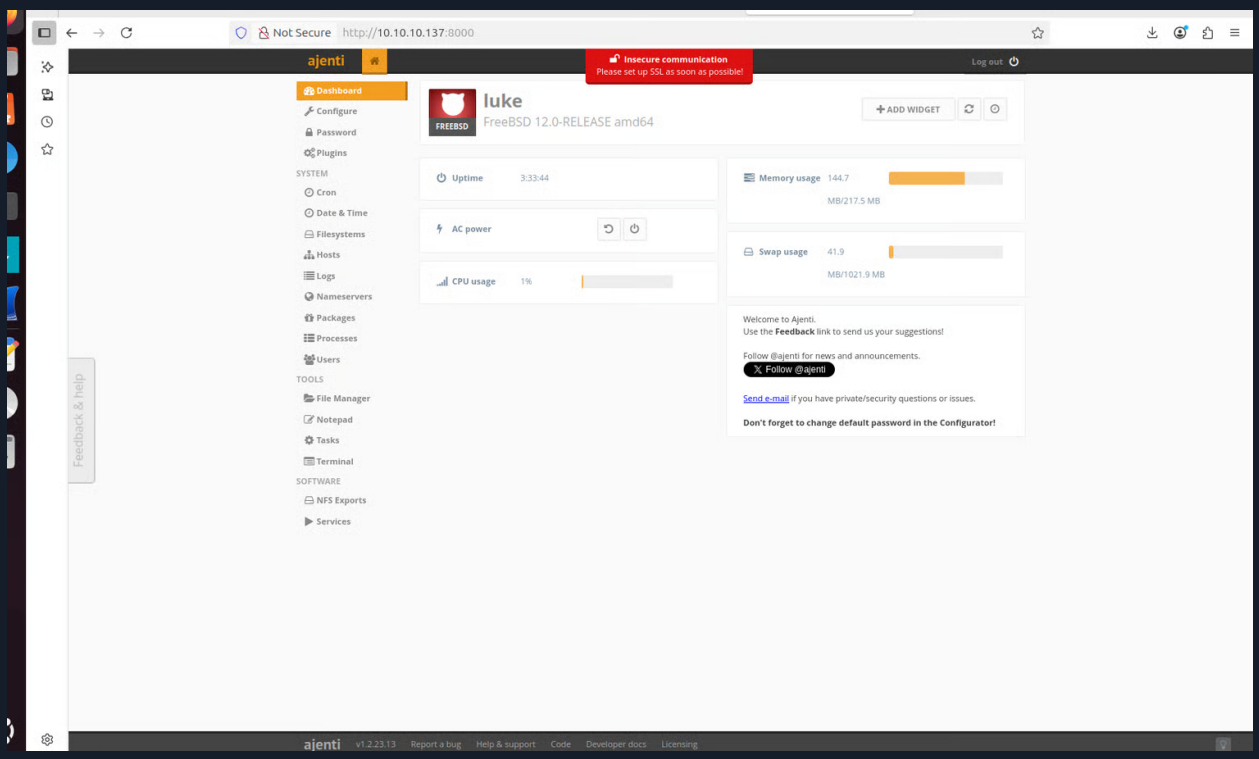


3. Click on `config.json` to view its contents. The JSON structure reveals the configuration for Ajenti, specifically the `users` object, which contains the `root` user and its plaintext password:

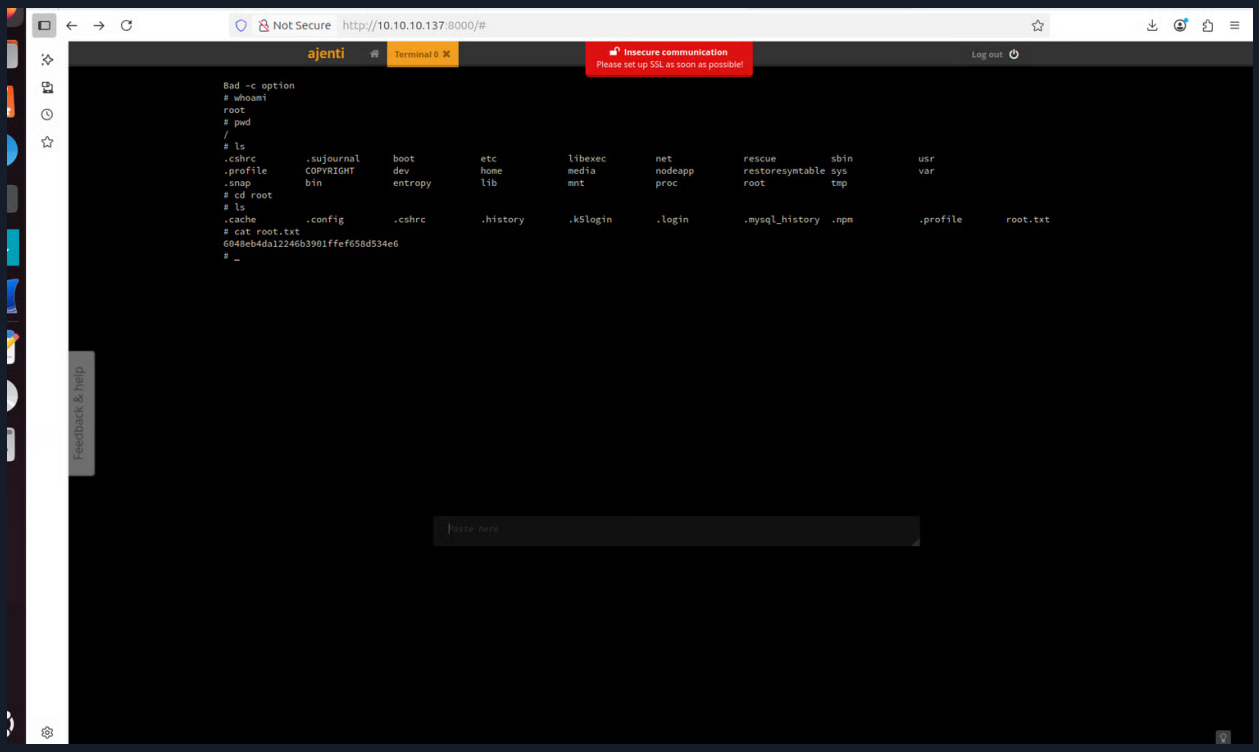
```
"users": {
    "root": {
        "password": "KpMaswqhS5ETTy9Z",
        "user": "root",
        "hostname": "localhost"
    }
}
```



4. Navigate to the Ajenti web interface on port 8000 (`http://10.10.10.137:8000`) and authenticate using the newly discovered credentials (`root` / `KpMaswqhS5ETTy9Z`).
5. Authentication is successful, granting access to the Ajenti dashboard.



6. From the Ajenti dashboard, navigate to the "File Manager" or "Terminal" tools. Because the application is running with elevated privileges, you can navigate directly to the root directory `/root` and read the `root.txt` flag, confirming full system compromise.


Not Secure http://10.10.10.137:8000/#
ajenti
Terminal 0
Insecure communication
Please set up SSL as soon as possible!
Log out
Bad -c option
whoami
root
pwd
/
ls
.cshrc .sujournal boot etc libexec net rescue sbin usr
.profile COPYRIGHT dev home media nodeapp restoresymtable sys var
.snap bin entropy lib mnt proc root tmp
cd root
ls
.cache .config .cshrc .history .k5login .login .mysql_history .npm .profile root.txt
cat root.txt
6048eb4da12246b3901ffef658d534e6
_
Feedback & help
Paste here

2. Hardcoded Credentials in Web Application Source Code - High

CWE	CWE-798 - Use of Hard-coded Credentials
CVSS 3.1	7.5 / CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N
Root Cause	**Vulnerability Description** The web application hosted on port 80 contains a publicly accessible configuration file (`config.php`) that exposes hardcoded database credentials. Additionally, it appears the web server is misconfigured to serve the raw source code of `.php` files rather than executing them, allowing unauthenticated users to view the backend code. Reviewing the source of this file revealed the plaintext username (`root`) and a highly complex password.
Impact	**Impact** Storing credentials directly within source code or publicly accessible configuration files completely bypasses authentication controls. An unauthenticated attacker can easily extract these credentials and utilize them to attempt lateral movement or privilege escalation. In this environment, the exposed password was reused, which directly facilitated unauthorized access to an internal API on port 3000.
Remediation	**Remediation** • **Remove Hardcoded Secrets:** Immediately remove all hardcoded passwords, API keys, and database credentials from the application's source code. • **Secure Storage:** Store sensitive configuration variables in secure environment variables or utilize a dedicated secrets management vault. • **Server Configuration:** Correct the web server configuration to ensure that `.php` files are properly parsed and executed by the PHP engine, preventing the raw source code from being returned to the client. Alternatively, move all configuration files outside of the public web root (`/var/www/html/` etc.) so they cannot be directly requested.
References	-

Finding Evidence

Technical Details

An attacker can utilize a directory fuzzing tool to discover hidden files and directories on the web server.

1. Run a directory enumeration tool against the target web server on port 80:

```
ffuf -w /path/to/seclists/Discovery/Web-Content/directory-list-2.3-medium.txt -u http://
10.10.10.137/FUZZ -mc 200-400
```

2. The output reveals a `config.php` file. Navigate to this file in a web browser.

3. Use the browser's "View Page Source" functionality (or utilize `curl`) to view the raw contents of the file, which exposes the following database credentials:

```
$dbHost = 'localhost';
$dbusername = 'root';
$dbPassword = 'Zk6heYCyv6ZE9Xcg';
```

3. Broken Access Control and Sensitive Data Exposure via API - Medium

CWE	CWE-200 - Exposure of Sensitive Information to an Unauthorized Actor
CVSS 3.1	6.5 / CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N
Root Cause	**Vulnerability Description** The web service operating on port 3000 exposes an administrative API. During the assessment, it was discovered that the `/login` endpoint is vulnerable to credential stuffing/reuse, accepting the hardcoded database password discovered previously. More critically, once authenticated, the `/users` endpoint fails to enforce proper object-level authorization and returns excessive sensitive data. Querying this endpoint returns a full JSON array of all registered application users, including their roles and plaintext passwords.
Impact	**Impact** Exposing plaintext passwords via an API response is a severe security flaw. An attacker who gains any level of authenticated access to this API can immediately harvest the credentials of all other users on the system, including administrators. This allows for horizontal and vertical privilege escalation, account takeover, and further lateral movement across the infrastructure.
Remediation	**Remediation** • **Data Minimization:** Refactor the API to strictly filter the data returned in its responses. API endpoints should never return sensitive data such as passwords, password hashes, or security tokens in their JSON responses. • **Implement Proper Authorization:** Enforce strict Role-Based Access Control (RBAC) and Object-Level Authorization. Ensure that an authenticated user can only query and view data they are explicitly authorized to access. • **Secure Password Storage:** Ensure that backend systems store passwords using strong, modern hashing algorithms (e.g., Argon2, bcrypt) with unique salts, rather than in plaintext.
References	-

Finding Evidence

Technical Details

An attacker can exploit this vulnerability by authenticating to the API and querying the vulnerable endpoints using the resulting JSON Web Token (JWT).

1. Send a POST request to the `/login` endpoint using the credentials discovered in `config.php`, slightly modified to use the `admin` username:

```
curl -X POST http://10.10.10.137:3000/login -d "username=admin&password=Zk6heYCyv6ZE9Xcg"
```

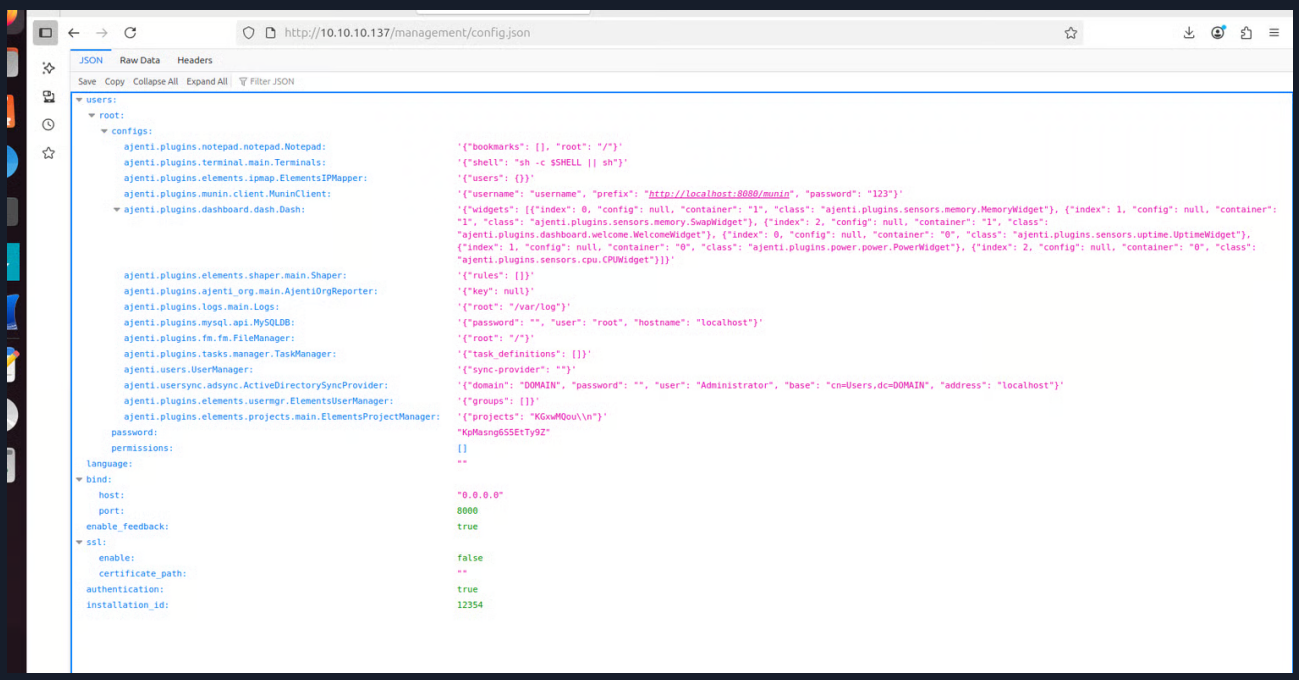
The server responds with a valid Authorization token.

2. Utilizing the obtained token, send a GET request to the `/users` API endpoint:

```
curl -s http://10.10.10.137:3000/users -H 'Authorization: Bearer [paste_token_here]'
```

3. The server responds with an array of all users and their plaintext passwords:

```
{"success":true,"message":"Authentication successful!","token":"eyJhbGciOi..."}
[{"ID":"1","name":"Admin","Role":"Superuser"},{"ID":"2","name":"Derry","Role":"Web Admin"},...]
```



4. Direct queries to specific users also reveal their passwords in plaintext:

```
curl -s http://10.10.10.137:3000/users/Admin -H 'Authorization: Bearer [paste_token_here]'
```

Output:

```
{"name":"Admin","password":"WX5b7)>rP$U)FW"}
```

4. Anonymous FTP Access and Information Disclosure - Medium

CWE	CWE-552 - Files or Directories Accessible to External Parties
CVSS 3.1	5.3 / CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N
Root Cause	The FTP service running on port 21 is misconfigured to allow anonymous authentication. The product makes files or directories accessible to unauthorized actors, even though they should not be. This allows any remote user to connect and authenticate to the server without providing a password or unique credentials. During the assessment, it was discovered that an attacker can log in using the username `anonymous` and access internal files, specifically a text file named `for_chihiro.txt`.
Impact	While the immediate impact of the exposed file was limited to information disclosure, allowing anonymous FTP access provides an attacker with a foothold to gather internal intelligence. The negative technical impact that arises if an adversary succeeds in exploiting this weakness is the ability to read files or directories. This intelligence can often be chained with other vulnerabilities to further compromise the system.
Remediation	Disable anonymous login functionality within the FTP server's configuration file. Ensure that all FTP access requires valid, unique credentials. Access control permissions should be set to prevent reading of sensitive files. Access control involves the use of protection mechanisms such as authentication to prove the identity of an actor.
References	-

Finding Evidence

Technical Details

An attacker can initiate a connection to the target FTP server and authenticate using the `anonymous` username and a blank password.

1. Connect to the target using the FTP command-line client:

```
ftp 10.10.10.137
```

2. When prompted for a name, enter `anonymous` and press enter for the password field:

```
Name (10.10.10.137:root): anonymous
331 Please specify the password.
Password:
230 Login successful.
```

3. Once authenticated, list the directory contents and download the exposed file to the local machine:

```
ftp> ls
ftp> get for_chihiro.txt
```

A Appendix

A.1 Finding Severities

Each finding has been assigned a severity rating of critical, high, medium, low or info. The rating is based off of an assessment of the priority with which each finding should be viewed and the potential impact each has on the confidentiality, integrity, and availability of Luke's data.

Rating	CVSS Score Range
Critical	9.0 – 10.0
High	7.0 – 8.9
Medium	4.0 – 6.9
Low	0.1 – 3.9
Info	0.0

A.2 Host & Service Discovery

IP Address	Port	Service	Notes
TODO FILL IN AS APPROPRIATE			

A.3 Subdomain Discovery

URL	Description	Discovery Method
TODO FILL IN DISCOVERED VHOSTS/SUBDOMAINS		

A.4 Exploited Hosts

Host	Scope	Method	Notes
TODO FILL IN AS APPROPRIATE	Text	Text	Text

A.5 Compromised Users

Username	Type	Method	Notes
TODO FILL IN AS APPROPRIATE	Text	Text	Text

A.6 Changes/Host Cleanup

Host	Scope	Change/Cleanup Needed
TODO FILL IN AS APPROPRIATE		

A.7 Flags Discovered

Flag #	Host	Flag Value	Flag Location	Method Used
1.	TODO HOSTNAME	TODO MD5 HASH	TODO Web root	TODO Unrestricted file upload (example)
2.				
3.				
4.				
5.				
6.				
7.				
8.				
9.				
10.				
11.				
12.				
13.				

End of Report

This report was rendered
by SysReptor with
♥